

LeetCode Pattern Recognition

How to identify the right algorithm from constraints & complexity

■ Fresher Interview Prep

■ Pattern-First Thinking

■ Constraint → Algorithm Map

SECTION 1 — Why Constraints Are Your Best Friend

Before writing a single line of code, read the constraints. They reveal EXACTLY which algorithm fits. Constraints tell you the allowed time complexity, which directly maps to a known pattern.

INPUT SIZE (N) → EXPECTED TIME COMPLEXITY → PATTERN TO USE

N (Input Size)	Max Allowed T.C.	Go-To Pattern / Algorithm
$N \leq 10$	$O(N!)$ or $O(2^N)$	Brute Force, Backtracking, Permutations
$N \leq 20$	$O(2^N)$	Bitmask DP, Subset Enumeration
$N \leq 100$	$O(N^3)$	3-pointer, Floyd-Warshall, DP (3D)
$N \leq 1,000$	$O(N^2)$	Bubble/Insertion Sort, 2D DP, Brute Force
$N \leq 10,000$	$O(N^2)$ tight	Two Pointers, Sliding Window w/ nested loop
$N \leq 100,000$	$O(N \log N)$	Sorting, Binary Search, Merge Sort, Heap
$N \leq 1,000,000$	$O(N)$ or $O(N \log N)$	Hash Map, Prefix Sum, Two Pointer, BFS
$N \leq 10^8$	$O(N)$ or $O(\log N)$	Math, Bit Manipulation, Binary Search only
$N \leq 10^{18}$	$O(\log N)$	Fast Power, Number Theory, Matrix Exponentiation

■ TIP: If $N=10^5$, $O(N^2) = 10^{10}$ operations → TLE. Always check 10^8 ops $\approx$ 1 second rule.

SECTION 2 — Space Complexity Clues in Constraints

Space constraints in the problem tell you WHICH DATA STRUCTURES you're allowed to use.

Space Constraint / Clue	Meaning	Pattern Suggested
"O(1) extra space"	No auxiliary arrays/maps	Two Pointers, In-place ops, Math
"Modify input in-place"	Cannot use extra array	Swap-based, cyclic sort, reverse
"Use O(N) space"	Can use 1 extra DS	HashMap, Stack, Queue, DP array
"Cannot use extra memory"	Strict O(1)	Floyd's cycle, XOR tricks, Moore's voting
"Return as array"	Output space doesn't count	Prefix sum, sliding window output
Memory limit = 256MB	~64M integers allowed	2D DP tables, adjacency matrix OK
Memory limit = 64MB	~16M integers	Avoid $N \times N$ matrices for $N > 4000$

■ **WATCH OUT:** Recursion stack = $O(\text{depth})$ space! A DFS on a tree of depth N uses $O(N)$ space even with no extra variables. If $O(1)$ space is required → convert recursion to iteration using an explicit stack.

SECTION 3 — Problem Keywords → Pattern Mapping

Scan the problem statement for these trigger words to instantly identify the pattern:

Keyword / Phrase in Problem	Pattern to Use	Example Problems
"Subarray / Contiguous / Sum"	Sliding Window / Prefix Sum	Max subarray, Subarray sum=K
"Sorted array + Target"	Binary Search / Two Pointers	Two Sum II, Search in rotated
"Shortest path / Min cost"	BFS (unweighted) / Dijkstra	Word ladder, Network delay
"All combinations / Subsets"	Backtracking / Bitmask	Subsets, Combination Sum
"Palindrome"	Two Pointers / DP	Longest palindromic substring
"Top K / Kth largest"	Heap (PriorityQueue)	K closest, Top K frequent
"Linked List cycle / middle"	Fast & Slow Pointers	Detect cycle, Middle of list
"Binary Tree / BST"	DFS / BFS / Inorder	LCA, Level order traversal
"Graph / Connected components"	Union-Find / BFS / DFS	Number of islands, Course sched
"Intervals / Overlap / Merge"	Sort + Greedy	Merge intervals, Meeting rooms
"Max / Min of overlapping windows"	Monotonic Deque	Sliding window maximum
"Number of ways / Count paths"	Dynamic Programming	Climb stairs, Unique paths
"String matching / Anagram"	HashMap / Sliding Window	Find anagrams, Minimum window
"Parentheses / Valid expression"	Stack	Valid parens, Evaluate RPN
"Next greater element"	Monotonic Stack	Daily temperatures
"Matrix / Grid / 2D"	BFS / DFS / DP on grid	Rotting oranges, Unique paths II
"Trie / Prefix / Autocomplete"	Trie (Prefix Tree)	Word search II, Implement Trie

SECTION 4 — The 5-Step Pattern Recognition Framework

STEP 1	READ CONSTRAINTS FIRST Check N (input size). Pick max allowed time complexity using the table in Section 1. Eliminate patterns that don't fit.
STEP 2	IDENTIFY KEYWORDS Scan problem text for trigger words (Section 3). Mark ALL keywords before deciding — one problem can have multiple clues.
STEP 3	CHECK SPACE RESTRICTIONS Look for "in-place", " $O(1)$ space", or memory limits. This narrows your data structure choices.
STEP 4	MATCH EXAMPLE WALKTHROUGH Walk through the given example manually. Ask: Is this a search? A count? A transformation? A reachability problem?

STEP 5

VERIFY WITH EDGE CASES

Check: empty input, N=1, all same elements, negative numbers. A pattern that fails edge cases = wrong pattern.

SECTION 5 — Top 14 Patterns: When to Use & Complexity

Pattern	Use When	Time	Space	Java Hint
Sliding Window	Subarray/substring, fixed or variable window	$O(N)$	$O(1) - O(K)$	Two int pointers l, r
Two Pointers	Sorted array, pair finding, palindrome	$O(N)$	$O(1)$	$l=0, r=n-1$; while $l < r$
Fast & Slow Ptr	Linked list cycle, middle, duplicate	$O(N)$	$O(1)$	$slow=slow.next$; $fast=fast.next.next$
Binary Search	Sorted data, search space reduction	$O(\log N)$	$O(1)$	while($lo \leq hi$) $mid=lo+(hi-lo)/2$
BFS	Shortest path, level order, min steps	$O(V+E)$	$O(V)$	Queue<> q = new LinkedList<>()
DFS/Backtracking	Paths, combinations, permutations, maze	$O(2^N)$	$O(N)$	Recursive + visited[] or used set
Dynamic Prog.	Overlapping subproblems, optimal substructure	$O(N^2)$	$O(N) - O(N^2)$	int[] dp = new int[n+1]
Heap/PriorityQ	Top K, Kth largest, merge K sorted	$O(N \log K)$	$O(K)$	PriorityQueue<> pq = new PriorityQueue<>()
Union-Find	Connected components, dynamic connectivity	$O(N \log N)$	$O(N)$	int[] parent = new int[n]; find+union
Monotonic Stack	Next greater/smaller, temperature, histogram	$O(N)$	$O(N)$	Deque<Integer> stk = new ArrayDeque<>()
Prefix Sum	Range sum queries, subarray sum = k	$O(N)$	$O(N)$	prefix[i] = prefix[i-1] + nums[i]
Trie	String prefix, autocomplete, word search	$O(L)$	$O(N * L)$	TrieNode with children[26]
Bit Manipulation	XOR tricks, subset via bitmask, unique	$O(N)$	$O(1)$	$a^b, n \& (n-1), 1 \leq i$
Greedy	Locally optimal $\rightarrow$ globally optimal, interval scheduling	$O(N \log N)$	$O(1) - O(N)$	Sort + iterate with condition

SECTION 6 — Read the Problem $\rightarrow$ Spot the Pattern (Worked Examples)

Example 1 — LeetCode #3: Longest Substring Without Repeating Characters

Constraints:

$0 \leq s.length \leq 5 \times 10^4$ | s has English letters, digits, symbols

Pattern Clues:

- ✓ $N = 50,000 \rightarrow O(N)$ or $O(N \log N)$ needed
- ✓ Keyword: "substring" $\rightarrow$ Sliding Window
- ✓ No space restriction $\rightarrow$ HashMap allowed

 $\rightarrow$ PATTERN:

Sliding Window + HashMap

Complexity:

Time: $O(N)$ | Space: $O(\min(N, \text{charset}))$

Java Template:

```
int l=0, max=0; Map map=new HashMap<>();
for(int r=0;r<s.length;r++){
    map.put(s[r], r);
    if(map.containsKey(s[l])){
        l=map.get(s[l])+1;
    }
    max=Math.max(max, r-l+1);
}
```

Example 2 — LeetCode #200: Number of Islands

Constraints:

$1 \leq m, n \leq 300$ | `grid[i][j]` is '0' or '1'

Pattern Clues:

- ✓ $N=300 \times 300=90,000 \rightarrow O(N \times M)$ needed
- ✓ Keywords: "grid", "connected" $\rightarrow$ BFS/DFS
- ✓ No extra space limit $\rightarrow$ visited array OK

$\rightarrow$ PATTERN:

DFS or BFS on Grid

Complexity:

Time: $O(M \times N)$ | Space: $O(M \times N)$ for recursion

Java Template:

```
void dfs(char[][] g, int r, int c){
    if(r<0||r>=m||c<0||c>=n||g[r][c]=="0") return;
    g[r][c]="0"; // mark visited
    dfs(g,r+1,c); dfs(g,r-1,c);
    dfs(g,r,c+1); dfs(g,r,c-1);
}
```

Example 3 — LeetCode #295: Find Median from Data Stream

Constraints:

$-10^5 \leq \text{num} \leq 10^5$ | Up to 5×10^4 calls

Pattern Clues:

- ✓ Continuous insertion $\rightarrow$ cannot sort repeatedly
- ✓ Keyword: "median" $\rightarrow$ need lower half & upper half
- ✓ $O(\log N)$ insert needed $\rightarrow$ Heap

$\rightarrow$ PATTERN:

Two Heaps (Max-Heap + Min-Heap)

Complexity:

Time: $O(\log N)$ per `addNum`, $O(1)$ for `findMedian` | Space: $O(N)$

Java Template:

```
PriorityQueue lo = new
PriorityQueue<>(reverseOrder());
PriorityQueue hi = new PriorityQueue<>();
// lo = max-heap (left half), hi = min-heap (right
half)
```

SECTION 7 — Quick Pattern Decision Guide

Question to Ask	YES $\rightarrow$	NO $\rightarrow$
Is the input SORTED or can you sort it?	Binary Search / Two Pointers	Check next $\rightarrow$
Does it involve a SUBARRAY or SUBSTRING?	Sliding Window / Prefix Sum	Check next $\rightarrow$
Does it need SHORTEST PATH or MIN STEPS?	BFS (unweighted) / Dijkstra	Check next $\rightarrow$
Does it need ALL COMBINATIONS / PATHS?	Backtracking / DFS	Check next $\rightarrow$
Are there OVERLAPPING SUBPROBLEMS / CHOICES?	Dynamic Programming	Check next $\rightarrow$
Is it a TREE or GRAPH traversal?	DFS / BFS / Union-Find	Check next $\rightarrow$
Need TOP K or K-TH ELEMENT dynamically?	Heap (PriorityQueue)	Check next $\rightarrow$
Is there a CYCLE or need CONNECTED components?	Floyd's / Union-Find	Check next $\rightarrow$
NEXT GREATER/SMALLER element problem?	Monotonic Stack	Consider Math / Greedy

SECTION 8 — Common Pattern Mistakes & How to Avoid

Mistake	Why It Happens	Fix
Using sort when $O(N)$ needed	Didn't check N vs complexity table	If $N > 10^5$, avoid $O(N \log N)$ sort first

Mistake	Why It Happens	Fix
Recursive DFS on large input → Stack Overflow	Stack Overflow = O(depth) stack space	Use iterative DFS with explicit stack
Using O(N) space when problem says 'in-place' constraint	Overlooking 'in-place' constraint	Read ALL constraint lines, not just N
BFS when DFS is sufficient	Confusing 'find if path exists' vs 'shortest path'	BFS = shortest. DFS = existence/all paths
DP when Greedy works	Overcomplicating; not checking greedy property	Check: does locally optimal = globally optimal?
Two Pointers on unsorted array	Two pointers needs sorted or specific structure	ALWAYS sort first OR verify structure
Prefix Sum without handling negative numbers	Forgetting prefix for subarray sum >= K	Use HashMap: map.put(prefix, index)

■ Remember: Constraints → Time Complexity → Pattern. Practice reading constraints BEFORE the problem description. After solving 50+ problems, pattern recognition becomes instinct. Good luck, Yuktha! ■